

Chapter 4

The Generation of Experimental Data for Computational Testing in Optimization

Nicholas G. Hall and Marc E. Posner

Abstract This chapter discusses approaches to generating synthetic data for use in scientific experiments. In many diverse scientific fields, the lack of availability, high cost or inconvenience of the collection of real-world data motivates the generation of synthetic data. In many experiments, the method chosen to generate synthetic data can significantly affect the results of an experiment. Unfortunately, the scientific literature does not contain general protocols for how synthetic data should be generated. The purpose of this chapter is to rectify that deficiency. The protocol we propose is based on several generation principles. These principles motivate and organize the data generation process. The principles are operationalized by generation properties. Then, together with information about the features of the application and of the experiment, the properties are used to construct a data generation scheme. Finally, we suggest procedures for validating the synthetic data generated. The usefulness of our protocol is illustrated by a discussion of numerous applications of data generation from the optimization literature. This discussion identifies examples of both good and bad data generation practice as it relates to our protocol.

4.1 Introduction

Most scientific models require the use of data for evaluation and validation. The three major sources that researchers use to obtain this data are the real world, library databanks, and random generation procedures. A natural choice is to use real-world data. One advantage of using this type of data is its relevance, which enables esti-

Nicholas G. Hall

Department of Management Sciences, The Ohio State University, Columbus, Ohio, USA

e-mail: hall_33@fisher.osu.edu

Marc E. Posner

Department of Integrated Systems Engineering, The Ohio State University, Columbus, Ohio, USA

e-mail: posner.1@osu.edu

mation of the eventual practical usefulness of the results of an experiment. A second advantage of using real-world data is that it enhances the credibility of the experiment. Moreover, this type of data is usually unbiased. However, there can be serious difficulties with using real-world data. First, it may not be available, or not available in quantities sufficient for the purposes of the experiment. This situation typically arises when a research area is new, or when the phenomenon being examined is fragile. An example of the latter situation is data collected from destructive testing. Further, most real-world data for a given application are generated by the same physical process. This may limit the size and variety of the data sets. For example, the amount of inventory data for a given product typically does not exceed the number of working days in a year. Moreover, each point is generated by the same underlying production and demand processes. There are also cases where very large and varied data sets are needed for testing (Wei et al. 2003). Further, there are situations where the collection of real-world data is costly, slow or inconvenient. Another concern is that, even when real-world data is available, it may be preferable to use data that is free from the stochastic vagaries of the real environment (Kadlec 2000). A further issue with real world data is that it can impose a very special, restricted structure on the problem (Fischetti et al. 2001).

There are many applications for which library data is available. Examples occur in the testing of computer systems and algorithms. Among the advantages of library data sets are that some properties of the problem, such as an optimal solution and the performance of other procedures on the data set, are known. Also, there is a recent literature on determining which data from an existing data set should be sampled to meet the objectives of a particular experiment (Koehler and Owen 1996). However, there are many applications where an adequate data library does not exist (McGeoch 1996). Another concern is that library data sets are sometimes biased by the inclusion of problems for which a particular previous experiment gives good results (Hooker 1995). A final problem is that “you get what you measure.” A comparison of procedures based on a set of library data leads to selection of procedures that work efficiently for that set. The selected procedures may work poorly for more typical types of data.

When it is not possible or desirable to use real-world or library data, an alternative is to generate *random* data. This type of data is also known as *synthetic* data. Generally, synthetic data is generated by a computer program using a pseudorandom number generator. Sometimes synthetic data is generated by perturbing real-world or library data. This protects confidentiality (Reiter 2002), introduces uncertainty into a deterministic problem (Bertsimas et al. 2006), and generates additional test data (Bienstock et al. 2006, Linderoth et al. 2006, Pan and Shi 2007). Some authors view synthetic data generation as a last resort (Degraeve and Schrage 1997). However, the substantial advantages of synthetic data include the speed and ease of generation, and the quantity, variety, and relevance of data that can be obtained. As an example of the latter, extreme problem instances that are needed to study robust performance can be generated efficiently using synthetic data (Yaman et al. 2007).

Applications of synthetic data generation can be found in many diverse scientific fields. Examples include: acoustics (Hooshyar et al. 2000), astrophysics (McIn-

tosh et al. 2000), analytical chemistry (Grate et al. 1999), cluster analysis (Krieger and Green 1999), computer imaging (Wilson and Hancock 2000), crystallography (Roversi et al. 1998), database design (Beyer et al. 1999), data mining (Shen et al. 1999), environmental engineering (Hooker 1995), evolutionary trees (Pearson et al. 1999), fuzzy systems (Ray and Ghoshal 2000), hydrology (Aksoy and Bayazit 2000), marketing (Bijmolt and Wedel 1999), medicine (Freed 2000), meteorology (Yuval 2000), mineral processing (Schena and Chiaruttini 2000), neural networks (Bauer et al. 1999), pattern recognition (Ho and Baird 1997), physics (Ruchala et al. 1999), radio science (Herique 1999), robotics (Gonzalez and Gutierrez 1999), seismology (Lu et al. 1997), statistical inference (Qin and Jing 2000), time series (Goutte 2000) and transportation planning (Munizaga et al. 2000). The frequency with which synthetic data is generated, the impressive variety of applications, and the considerable impact of the data generation process on experimental results all emphasize the importance of proper data generation methods. In spite of this, there are no general protocols for the generation of synthetic data.

Because of the lack of general guidelines, the scientific literature contains many examples where difficulties arise in synthetic data generation. For instance, in cluster analysis, standard random sampling techniques frequently miss small clusters (Palmer and Faloutsos 2000). Also, there is evidence that synthetic surface seismic data show a poor qualitative match with real data (Frenje and Juhlin 1998). As another example, most noise components generated in synthetic data are similar to Gaussian white noise, whereas the noise that occurs in real data typically follows a more complex and varied pattern (Gelius and Westerdahl 1997). Finally, as discussed in Hall and Posner (2001), there are experiments that test computer algorithms where the synthetic data contains instances for which the optimal values of most variables can be found trivially (Potts and Van Wassenhove 1988). Additional examples of difficulties with data generation schemes are discussed in Sect. 4.3.

A frequent goal of synthetic data generation is to generate a “typical” class of problems. However, for most optimization applications, such a class is hard to characterize. For many experiments, this goal may even be inappropriate. A better approach is to generate classes of problems with different characteristics and investigate how the performance of a procedure depends on these characteristics (Hooker 1994, Hall and Posner 2007). Moreover, the generation of synthetic data sets in a controlled way represents an important step towards a more scientific approach to testing optimization procedures. This approach to data generation is supported by the literature on “no free lunch theorems” (Schaffer 1994, Wolpert and Macready 1997). Such theorems show that, if a procedure achieves superior results on some types of instances, it must compensate with inferior performance on other types of instances. This suggests that matching procedures to types of instance results in better performance than applying a common method across all instances. This idea is demonstrated by Hall and Posner (2007), Smith-Miles (2008), Smith-Miles et al. (2009). It further suggests that an arbitrary synthetic data generation procedure provides little useful information about the relative performance of optimization procedures.

Synthetic data is needed for a variety of purposes that include hypothesis testing, evaluation, design improvement, optimization, and software testing. In *hypothesis testing*, we may wish to investigate which of several alternative hypotheses best explains a natural phenomenon, which of several competing drugs is most effective at fighting a specific disease, or which computer algorithm runs fastest. A second purpose is *evaluation*. For example, we may wish to observe the cooling pattern of a metal in an injection mold, the spread of environmental pollution, or the behavior of a meteorological phenomenon. In these situations, synthetic data may be needed to perform a simulation study. Another purpose is *design improvement*. For example, we may test components or prototype versions of a system, product or algorithm at intermediate design stages. The objective is to gain information that improves the final design. Such testing is often classified as a pilot study. For example, we may wish to test an algorithm to ensure that it is robust for a variety of types of data. Random data is often needed because the uniqueness of the design process results in a lack of real-world data. Examples of systems and products where this may occur include designing a manufacturing system, increasing the sensitivity of a medical diagnostic procedure, and improving the performance of a laser weld. Synthetic data is also useful for solving *optimization* problems. For example, one standard approach for solving stochastic programming problems is to generate scenario trees using synthetic data. These trees discretize the continuous space of probabilistic problem scenarios. The scenario trees define stochastic programming problem instances that are solved in order to provide an approximate optimal solution. *Software testing*, which is closely related to optimization, is another purpose of synthetic data. This type of testing ensures the correctness of a software program. For this application, synthetic data must be generated so that the correctness of each line of software code is tested.

This chapter focuses on the generation of synthetic data to test the performance of optimization heuristics and algorithms. The scope includes hypothesis testing, evaluation, design improvement, and optimization. While testing software programs that implement optimization procedures is of practical importance, it imposes very different requirements on the data that is generated. As a result, we do not consider this type of synthetic data generation.

The chapter is organized as follows. In Sect. 4.2, we propose a general protocol for synthetic data generation. Sect. 4.3 discusses numerous applications of synthetic data generation for the testing of optimization algorithms. These applications illustrate both good and bad practices of synthetic data generation as it relates to our protocol. Sect. 4.4 provides a conclusion and some suggestions for future research.

4.2 A Protocol

A *data generation scheme* is a complete step-by-step specification of the procedure by which data is generated, including the parameter ranges and the random number generation method. Our protocol for synthetic data generation is a multilevel struc-

ture that consists of *generation principles*, *features of the application*, *features of the experiment*, *generation scheme*, and *validation*. The generation principles provide high-level perspectives on the data generation process and are independent of the details of a particular application or experiment. To provide more detailed guidelines that ensure the principles are satisfied, we develop *generation properties* to operationalize each generation principle. With the properties as guidelines for good practice, the features of the application and the features of the experiment determine the actual generation scheme. After the data is generated, we examine the *attributes* of the data to determine whether the data is suitable. To accomplish this, we must be able to quantify and measure these attributes. An overview of the protocol we propose appears in Fig. 4.1. The structure that we develop is used to discuss and evaluate generation schemes in Sect. 4.3.

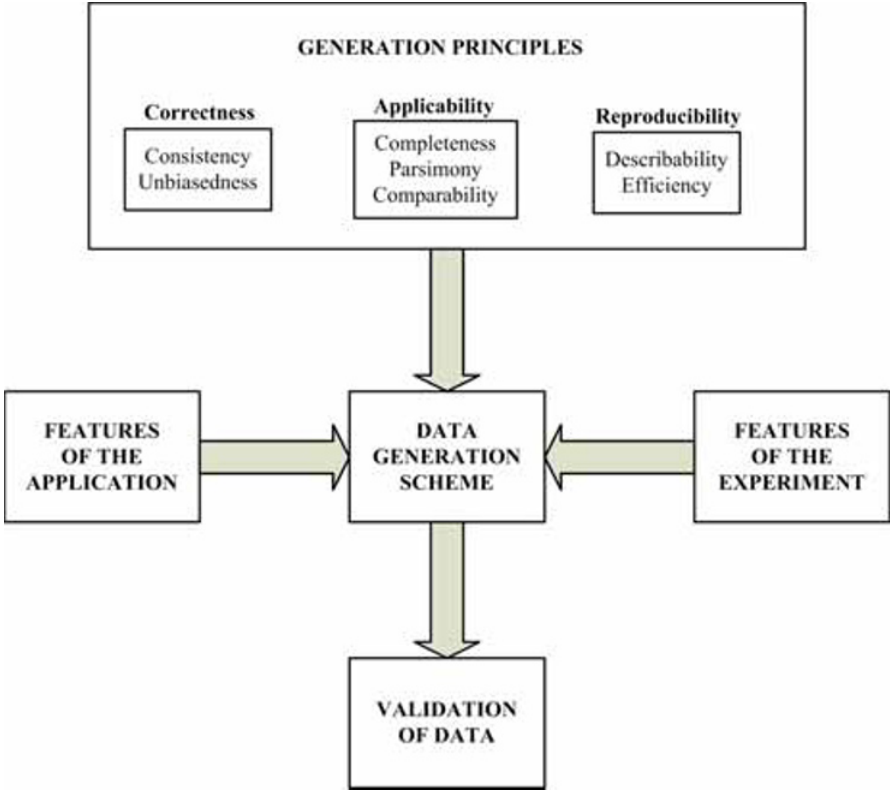


Fig. 4.1: A protocol for synthetic data generation

4.2.1 *Generation Principles and Properties*

Generation principles are high-level policies that should be followed when generating synthetic data. To operationalize these principles, we provide *generation properties*. These are more specific requirements that can be incorporated into any data generation scheme.

Three principles that should be present in a data generation scheme are:

1. **Correctness:** The data sets generated are free from defects.
2. **Applicability:** The generation scheme generates the types of data sets that are needed.
3. **Reproducibility:** The generation scheme and its data sets are reproducible.

It is important that the data is constructed in a way that is logical and free of distortions. Otherwise, the results of the analysis are not valid. Two generation properties that operationalize the principle of Correctness are:

- i. **Consistency:** Identical types of data are generated in the same way.
- ii. **Unbiasedness:** All biases in the data are controlled.

The Consistency property requires that all elements of the same type of data are generated in a similar way. Thus, the generation scheme should not provide different treatment for particular elements of the data. For example, assume that the mean of a particular element has a desired value. Consider a scheme that randomly generates data for all but the last element and then sets the value of the last element to achieve the desired mean. This procedure treats the last element differently from the others. Consequently, it is inconsistent and may have unpredictable and biased effects on the results of an experiment. A more consistent way to generate the values of the elements is to set an expected mean for the elements and allow some variance around that value (Hall and Posner 2001).

The Unbiasedness property requires that the data is generated in a way that does not promote some experimental results over others. In testing the relative performance of two or more algorithms, for example, certain characteristics of the data may favor a particular algorithm. Similarly, when evaluating a heuristic, the use of biased data may lead to erroneous conclusions. To formalize Unbiasedness, it is helpful to regard all possible data sets that can be generated by a particular synthetic data generation scheme as members of a population. The generation of a particular data set is analogous to the selection of a sample from this population. Then, Unbiasedness can be seen as a requirement that the sample is a fair representation of the scenarios being modeled. When constructing synthetic data, it is important not to introduce inadvertent biases into the generation scheme. However, this is not always possible. For example, as mentioned by Hadjar et al. (2006), the special structure of some real-world vehicle routing problems is hard to reproduce synthetically. Hence, the generated data may be biased.

The second generation principle is Applicability. The purpose of the experiment, such as hypothesis testing, evaluation, design improvement or optimization, may

greatly affect the way in which the data is generated. The Applicability criterion requires that the data is generated to satisfy this purpose. For example, suppose an experiment involves testing an algorithm over specific data sets that are likely to occur in practice. Then, the data generation scheme should produce data sets that are representative of these particular situations. Another example is a comparison between two algorithms. This experiment needs a varied set of data that represents all possible data sets to which these algorithms could be applied. As another example, complex interactions between components in the design of a heuristic procedure may generate issues that a designer needs to address. The generation of customized synthetic data that models these interactions may aid in the resolution of those issues. A final example occurs in generating a scenario tree for stochastic programming. To find solutions that are close to optimal, the scenario tree should discretize the solution space to a reasonable degree of accuracy, but be of a tractable size.

Three generation properties that operationalize the principle of Applicability are:

- i. Completeness: All data sets that are important to the experiment can be generated.
- ii. Parsimony: The variations in the data sets are important to the experiment.
- iii. Comparability: The experiments are comparable within and between studies.

It is important that a generation scheme has the property of Completeness. A good generation scheme can generate all data sets that are relevant to the specific scientific experiment. The amount of variety needed in the data is normally determined by the objectives of the experiment. If the objective is to obtain good results for specific types of data, then little variety is necessary. However, if the objective is to design an algorithm that is robust, i.e., is expected to perform well for a wide range of scenarios, then much greater variety is necessary. Also, a generation scheme that fails to vary important causative variables appropriately may lead to the statistical phenomenon called autocorrelation (Hays 1973). In this case, the real relationships between the variables, as well as the significance of randomness in those relationships, may become impossible to estimate. To test the specific behavior of a procedure, different types of data sets are sometimes needed. In this case, the generation scheme should be sufficiently robust to generate all of the various types. However, if different generation schemes are used, then there may be issues with Comparability, as discussed below. If a goal is to use a procedure in a specific real-world setting, then the synthetic data should be similar to the real-world data. Therefore, the generation process should, if possible, emulate the way in which data is generated in the relevant real-world application.

Parsimony is a property that eliminates spurious variation within synthetic data. In general, the variety that occurs in a synthetic data set arises from only two sources: systematic variation of parameters as part of the experimental design, and randomness. Parameter variation that can have no effect on the experiment may distort the perceived effects of experimental variation. This can falsify the results. The Parsimony property is frequently important when a hypothesis is being tested through parametric changes to causative variables. An example is the testing of a branch-and-bound algorithm for integer programming. Doubling the size of all pa-

rameters in the problem leaves the problem unchanged. A generation procedure that considers parameter ranges which grow proportionately for all parameters simultaneously creates additional sets with the same type of data.

It is frequently important to be able to compare experiments. A lack of Comparability within a study may lead to erroneous or questionable conclusions. For example, due to problems in converting real-world data into homogeneous synthetic data, Lu et al. (1997) uses different procedures to construct data for small and large earthquakes. While this does not create difficulties when analyzing either the small or the large earthquakes separately, comparative statements about the two types of earthquakes become more problematic. Achieving Comparability with earlier studies is also important for the soundness of the experimental conclusions. An earlier study may be based on real-world data that is either no longer available, is already studied exhaustively, or is too small for current purposes. In this case, we may want to emulate the generation process of the earlier real-world data. Particular care is also needed when testing alternative hypotheses. A conclusion that one algorithm is preferred over another for a particular set of data should not be extrapolated to conclude that this preference holds over all possible data sets (Schaffer 1994, Wolpert and Macready 1997).

The principle of Reproducibility has long been recognized as important for scientific verification. When researchers want to validate results by reproducing experiments, they must be able to generate the same types of data that were originally generated. Usually, synthetic data is not described by a full specification of the data sets generated. Instead, a description of the underlying generation scheme is provided that is sufficiently detailed to permit reproduction. The conciseness of such descriptions is a valuable advantage of synthetic data (Hall and Posner 2001). Of course, the same synthetic data generation scheme can randomly generate sets of test problems with very different characteristics, particularly if sample sizes are small. Therefore, Reproducibility requires that the pseudorandom number generation scheme and random number seeds be part of the documentation.

Two generation properties that operationalize the principle of Reproducibility are:

- i. Describability: The generation scheme is easy to describe.
- ii. Efficiency: The generation scheme is easy and efficient to implement, use, and replicate.

For a data generation scheme to be describable, the description of the data generation scheme should be complete and comprehensible. If it is not, then researchers are unable to replicate the study. This property is typically more important when the experiment is not unique and when the results will be published. This is because uniqueness or confidentiality implies that it is less likely that there will be subsequent comparisons or evaluations of the experiment.

Efficiency can be measured either quantitatively or qualitatively and tends to have many possible levels of success. The goal of Efficiency is to develop a generation scheme that is as easy as possible to implement, use, and replicate. This is particularly important when large data sets are needed. For large data sets, a useful

characteristic of synthetic data generation is the ability to link the data generation and experimentation stages so that it is necessary to store only one data set at a time. Ease of replication is also important for experiments that require large numbers of trials. For example, consider an experiment where randomly generated data sets from the same scheme have widely differing results. Then, many data sets may be needed to ensure the statistical significance of the conclusions. To validate these conclusions, statistical methods that determine the necessary sample size may be needed.

4.2.2 *Features of the Model*

Two important factors that determine which properties are important in a particular generation scheme are the *features of the application* and the *features of the experiment*. The features of the application are naturally occurring characteristics of the problem. These include the size of the data requirements, functional relationships between different parts of the data, and any bounds on the data. The features of the experiment are selected by the experimenter. These include the number and variety of tests, as well as intentions about implementation and publication. The features of the experiment may interact with the generation properties in various ways. For example, if only a single design is being evaluated, then Unbiasedness needs to be interpreted more narrowly. As another example, an algorithm may be designed for use in an application that is newly emerging and not yet well specified. For this application, the property of Completeness has less value, because it is unclear which data sets might be important to the experiment. As a final example, if the study is intended for commercial use and is not to be published, then Describability is less important.

Consequently, each generation property is relevant to a greater or lesser degree for a given experiment. As a result, we use the features of the model and experiment to select and modify the properties in order to guide the creation of the synthetic data generation scheme.

4.2.3 *Validating the Data*

A critical step in the data generation process is the validation of the synthetic data. One approach is to run a small preliminary study and check the reasonableness of the results. This approach is common when Monte Carlo simulation is used for experimental evaluation (Law and Kelton 1991). The main problem with this approach is that assessing reasonableness requires a prejudgment about the results of the experiment. An alternative is to compare the synthetic data against real-world data. However, as discussed above, a frequent reason for generating synthetic data is that

no real-world data is available. Also, if only a small amount of real-world data is available, then the results may not be statistically reliable.

Consequently, a better approach is to validate the synthetic data outside the structure of the experiment. To do so requires identifying the key *attributes* of the data that may influence the results of the experiment. For example, among the relevant attributes of homogeneous numerical data are its average value and its dispersion. These attributes need to be evaluated and controlled in order to conduct a meaningful and efficient experiment. The existence of particular attributes in the synthetic data is often important. Where relevant, the values of these attributes should be similar to those in the real-world data, and should also match the experimental design. Some attributes of the data may be hard to specify and measure. An example is where the relationship between the attributes and the experimental results is unclear or complex (for example, highly nonlinear). Another example is where the attributes are not easily quantifiable, for instance the degree of clustering.

An attribute may be evaluated for one or more of the following reasons:

1. To control a causative variable in order to perform a hypothesis test
2. To ensure that the data varies so as not to bias an experiment
3. To document data variation that occurs naturally in practice
4. As a proxy for another attribute that is harder to define or measure

Each of these reasons implies the need for a *measure* to evaluate the attribute. For example, the measures of mean and variance evaluate the attributes of average value and dispersion, respectively. These measures can be computed using standard statistical methods. In experiments that study a cause-and-effect relationship between one or more causative variables and a dependent variable, measures are needed to control the values of the causative variables. Furthermore, a measure is needed to ensure that a level of variety consistent with the objectives of the experiment is introduced into the synthetic data. Also, when there is an attempt to reproduce real-world variation in the data, a measure of that variation is needed.

Sometimes, the direct measurement of an attribute is computationally difficult. For example, computing the number of feasible solutions in an intractable optimization problem may itself be an intractable problem (Garey and Johnson 1979). Consequently, rather than measure the exact amount of computation required by an enumerative algorithm for the 0-1 knapsack problem, Hall and Posner (2007) develop a heuristic procedure for estimating the number of nodes in the search tree.

Currently, there is no literature on the identification of suitable measures of the attributes. Therefore, an *ad hoc* approach is usually necessary. Some characteristics that a measure should have include:

1. Relevance to the experiment
2. Change in an appropriate way
3. Tractability

First, the measure should quantify the attributes of a data set in a way that is relevant to the outcome of the experiment. For example, in the testing of production planning algorithms, a simple heuristic performs best on many kinds of synthetic

data. However, there are specific classes of data for which this is not true (Uma and Wein 1998). Thus, it would be valuable to determine and measure the important attributes of the data that could predict the performance of such a heuristic on a given data set. Second, the measure should not change spuriously with respect to the problem. For example, changing all costs proportionately in experimental testing of optimization software may leave the relative costs of all solutions unchanged. Consequently, the experimental results are unchanged also, and the measure should reflect this. Finally, since the purpose of a measure is to provide information about a data set, the evaluation of a measure should generally take significantly less time than it takes to perform the experiment itself.

As a result, for each important attribute, the data validation process should define one or more measures that can be used to determine to what extent the data contains that attribute. Comparisons between the synthetic data and any available real-world data are also valuable. For example, one relevant method is to test the hypothesis that the mean values of some attribute in the real data and the synthetic data are equal (Hays 1973). Finally, some experiments require several measures and these may be of different types. For example, if both computation time and accuracy are important to an experiment that tests the performance of a heuristic procedure then both attributes should be measured.

4.3 Applications to Optimization

In this section, we consider data generation issues that arise in various optimization applications. We identify several examples of good and bad data generation practice relative to the properties in our general protocol described in Sect. 4.2. Many of the data generation schemes we discuss are carefully designed, satisfy all of our criteria, and serve as excellent examples for future researchers to follow.

Unfortunately, in most synthetic data generation schemes, the generation properties are not all easy to satisfy. The Comparability property is usually easy to satisfy, except when both real-world and synthetic data are being used. Also, Describability is easy to satisfy in generation schemes for most optimization applications. An exception occurs in stochastic programming (see Sect. 4.3.7), where the underlying optimization problem formulations at each stage may need to be shown explicitly. Further, Efficiency is easy to satisfy for most optimization applications, except where large volumes of data are required. Stochastic programming is again an exception, due to the need to generate large scenario trees. Consequently, the four properties that require the most discussion in this section are Consistency, Unbiasedness, Completeness, and Parsimony.

A given generation property is easier to satisfy in some applications than in others. For example, data generation is easier when the required data has a simple structure that does not contain linking constraints or correlations. A second example occurs where real-world data has limited variation. This may imply that limited variation is also required in the synthetic data. A third example occurs where previous

experiments have already documented a well-designed data generation scheme that satisfies all of the properties. However, trade-offs may arise between two or more properties. For example, a data generation scheme that is designed for Efficiency may have difficulty satisfying the Unbiasedness property (Sherali and Smith 2006). As another example, the random network generator in Demeulemeester et al. (2003) represents a trade-off between generating the data sets efficiently and generating all possible graphs with equal probability. The latter is required for Completeness. Failure to achieve Completeness may introduce biases into the data. In such situations, the features of the experiment should be used to determine a suitable compromise between the generation properties.

In view of the comments in the previous paragraph, we recognize that, in some applications, designing a data generation scheme that satisfies all of the properties may be difficult or even impossible. Consequently, our critical comments that appear with respect to some of the data generation schemes discussed below should be interpreted as comparisons with an *ideal* data generation scheme that may be very hard to develop. Moreover, failure to satisfy one or more of the properties we propose does not, by itself, suggest that the results of a research study are invalid. Rather, it suggests that additional testing is needed to determine whether the shortcomings of the data generation scheme have influenced the results.

Sections 4.3.1–4.3.7 are organized by application types. However, Sect. 4.3.8 contains a discussion of data generation issues that arise from the intractability of optimization problems across various application types.

4.3.1 Generalized Assignment and Knapsack

Solution procedures for the generalized assignment problem are proposed and tested computationally by Ross and Soland (1975), Chalmet and Gelders (1976), Ross and Soland (1977), Martello and Toth (1981), Trick (1992), and Racer and Amini (1994). A stochastic model for this problem is developed by Romeijn and Morales (2001a). This model analyzes the data generation schemes proposed by the earlier authors. The analysis reveals that, in all of these prior works, the generation schemes produce data sets where the capacity constraints become less tight as the number of machines increases. Such behavior makes the resulting problem instances easier to solve. Consequently, using the generation schemes used by Ross and Soland (1975, 1977), it is possible to conclude that a proposed greedy heuristic almost always finds the optimal solution. However, the use of a data generation scheme that satisfies Consistency shows that increases in the number of machines result in small increases in the relative error of the heuristic (Romeijn and Morales 2001a).

The issue of correlation between subsets of the data requires discussion. For the 0-1 linear knapsack problem (Balas and Zemel 1980, Martello and Toth 1988, 1997), and some one-machine scheduling problems (Potts and Van Wassenhove 1988, 1992), the authors report significantly longer computation times when the objective and constraint coefficients are positively correlated. Also, Guignard and

Rosenwein (1989), Trick (1992), and Amini and Racer (1994) report the same phenomenon when the objective function coefficients and the capacity constraint coefficients in the generalized assignment problem are negatively correlated. However, several of these research studies use data generation schemes that do not control the amount of correlation between the parameters. For example, the generation scheme used by Martello and Toth (1979) for the knapsack problem considers only data that is uncorrelated and data that has a correlation coefficient above $+0.97$. For the generalized assignment problem, the study of Martello and Toth (1981) considers only correlation coefficients of 0 and below -0.97 . An issue with this type of data is that there is no information about how problem difficulty varies with correlation. An important question, which cannot be answered using the experimental designs in Martello and Toth (1979, 1981), is whether problem difficulty increases linearly with correlation or whether the increase occurs more quickly around some specific values of correlation. Other research studies that fail to control the correlation coefficient explicitly include those by Balas and Zemel (1980), Balas and Martin (1980), Potts and Van Wassenhove (1988), Guignard and Rosenwein (1989), John (1989), Rushmeier and Nemhauser (1993), and Amini and Racer (1994). Since the correlation coefficient strongly affects algorithmic performance, the computational studies in these works fail to satisfy the Completeness property. A procedure for approximately inducing prespecified Spearman rank correlation levels is proposed by Iman and Conover (1982). Also, a procedure for generating data with a prespecified Pearson correlation level between different subsets of data is developed by Hill and Reilly (1994). Both of these procedures are tested computationally on two-dimensional knapsack problems by Hill and Reilly (2000). They also find that correlation significantly affects the performance of both an optimal algorithm and a heuristic solution procedure. Because of the complexity of the procedure required to induce prespecified correlation levels, the Describability property is not easy to meet (Cario et al. 2002).

A study of a robust formulation of the 0-1 linear knapsack problem is considered by Atamtürk (2007). This study exclusively uses a knapsack size equal to half the item sizes. However, a study of the factors that affect computational difficulty in this problem shows that the knapsack capacity is one of the most important factors (Hall and Posner 2007). For this reason, it seems unlikely that the Completeness property can be satisfied without permitting the knapsack capacity to vary.

4.3.2 *Supply Chains*

A multiperiod single sourcing problem is considered by Romeijn and Morales (2001b). The authors derive a necessary and sufficient condition on the excess capacity such that the problem is feasible with probability one when the number of customers goes to infinity. This condition permits precise control of the tightness of the constraints in experimental data sets through the excess capacity parameter. Controlling the tightness of constraints as part of a generation scheme helps to sat-

isfy the Completeness property by generating a wide variety of data sets. It also helps to satisfy the Unbiasedness property because uniform tightness of constraints may favor particular types of procedures. Similar comments can be made about a combined sourcing and scheduling problem (Chen and Pundoor 2006).

A subgradient search algorithm for multi-item, multifacility supply chain planning is tested computationally by Wu and Golbasi (2004). The large variety of combinations of parameters tested clearly satisfies the Completeness property. The following procedure is used to generate random capacity values. In the first period, the total demand for the period is multiplied by a constant larger than one to generate a value for capacity. In later time periods, total capacity assigned for the previous time periods is subtracted from total cumulative demand, and the result is then multiplied by a constant to generate capacity. This procedure has the advantage of generating test problems with tight capacity constraints, which are the most useful problem instances for the computational experiment. However, since the capacity in one period depends on the demand for all previous periods, it is possible that the data generation scheme introduces unintended correlations. This can compromise the Unbiasedness property. Moreover, the random capacity is generated in the first period using a different procedure than is used in all subsequent periods. As a result, the generation scheme fails to satisfy the Consistency property.

In many practical situations, the times at which raw materials become available define time windows for production, as discussed in the context of a multiperiod lot-sizing problem (Brahimi et al. 2006). The testing of Lagrangian heuristics is performed using various parameter specifications for time window density, number of products, demand lumpiness, demand distributions, setup and holding costs, capacity tightness, and minimum and maximum time window lengths. The complex generation scheme satisfies the Consistency, Unbiasedness, Parsimony, and Comparability properties. However, clustering of the time windows is likely to affect problem difficulty and heuristic performance, since it results in resource bottlenecks. This raises concerns about whether the Completeness property can be satisfied without also varying this parameter. Further, for each period, the synthetic data generation procedure ensures that the cumulative aggregate demand is no larger than the cumulative total demand. Unfortunately, the details about how this is accomplished are not given. Thus, the data generation procedure lacks Describability. If the aggregate demand data is generated to fit the cumulative total demand, then this may fail to satisfy the Unbiasedness property. Moreover, if later aggregate demand is treated differently than earlier aggregate demand, then the Consistency property is not satisfied. A data generation process is more consistent if unacceptable aggregate demand streams are discarded and then new streams are generated to replace them. By varying only one parameter at a time, the authors simplify their experiment. However, this also eliminates potentially significant interaction effects (Hall and Posner 2007), which may compromise the Completeness property.

4.3.3 Scheduling

A procedure for the generation of random activity-on-node networks with resource constraints for project scheduling problems is proposed by Demeulemeester et al. (2003). A highly desirable but difficult to achieve feature of such networks is that they are strongly random, i.e., they can be generated at random from the space of all possible networks with specified numbers of nodes and arcs. The proposed procedure, *RanGen*, is the first random network generator that is able to control for two important predictors of project scheduling problem difficulty. It therefore provides an advantage with respect to the Completeness property over previous generators (Patterson 1984). A more specific advantage with respect to the same property is that networks with small densities can be generated, unlike with the previous generators by Kolisch et al. (1995) and Schwindt (1995). Furthermore, to speed up the generation process, some previously published network generators restrict the values of certain parameters that do not affect performance. Unfortunately, these restrictions compromise the randomness of the resulting networks, and violate the Parsimony property. However, *RanGen* satisfies the Parsimony property by avoiding these restrictions. Moreover, this procedure generates project networks that are “as strongly random as possible,” given a computation time constraint.

A methodology for data generation for scheduling problems is described by Hall and Posner (2001). The authors observe that release dates become clustered if many release dates are generated within a fixed interval that is independent of the total job processing time. In this situation, the jobs that arrive later do not have active due date restrictions, which fails to satisfy the Consistency property. Regarding due dates, generation schemes in which the earliest due date depends on all the processing times (Hariri and Potts 1983) fail to satisfy the Unbiasedness property. A further concern is that most of the variables may be trivially determined (Potts and Van Wassenhove 1988), in which case the Completeness property is not satisfied. Problems may also arise in the generation of precedence constraint graphs, where as the number of nodes increases, the expected density increases and becomes more varied over different parts of the graph (Potts and Van Wassenhove 1985, van de Velde 1995). This fails to satisfy the Consistency property. Another problem is that the generation of machine speeds from a uniform distribution (Ow 1985) may lead to many machines having similar speeds, which also fails to satisfy the Completeness property. For example, if ten machines have speeds 1, 2, ..., 10, respectively, then the ratio of the speeds of any pair of the last six machines is no larger than two. Improved data generation schemes that resolve these issues are described by Hall and Posner (2001). This work also satisfies all of the generation properties. In particular, the work proposes a precedence constraint generation scheme that both controls the expected density of the precedence constraint graph and also ensures that the expected density is equal across all nodes.

4.3.4 Graphs and Networks

The graph and network problem environment is simpler than many other optimization problems. This makes synthetic data generation easier, and reduces conflicts between the properties. One example of a justifiable data generation scheme is given by Laguna and Rafael (2001). The optimization problem studied involves coloring sparse graphs. Random test graphs are drawn from the class of random graphs with n vertices and edge probability p . Hence, each edge appears in the graph with probability p , independent of whether the other edges appear, which satisfies the Consistency property. The simple generation scheme proposed also satisfies the Unbiasedness, Parsimony, Comparability, Describability and Efficiency properties.

An interesting geometric optimization problem, finding the smallest circle that encloses a set of given circles, is studied by Xu et al. (2003). The centers of the circles in the data set are generated as normally distributed random points, and then the radii are generated as uniformly distributed random numbers. This generation scheme satisfies the Consistency, Unbiasedness, and Parsimony properties. The performance of four fundamentally different algorithms is compared in the computational experiment. These methods are second-order cone optimization, subgradient optimization, quadratic programming, and randomized incremental construction. Given these widely differing solution approaches, it may be appropriate to support the Completeness property by introducing additional variety into the generation scheme. For example, there could be clustering of the given circles, or correlation of the location and size of those circles. It seems likely that including such parameter variations would strengthen the results of the experiment.

A multiperiod capacity expansion problem on networks is considered by Bienstock et al. (2006). The authors use a carefully designed generation scheme that starts with five real-world networks, but varies several parameters randomly, resulting in the generation of 7,750 problem instances. The generation scheme incorporates several features motivated by real-world considerations, including decreasing capacity investment costs and increasing maintenance costs, over time. As a result, the Unbiasedness and Completeness properties are satisfied.

The study of the vertex packing problem by Sherali and Smith (2006) makes use of a formulation in which all rows have to be covered at least once. This is achieved by generating a matrix $A = \{a_{ij}\}$ of random coefficients, where $a_{ij} \in [-1, 1]$. These coefficients are then multiplied by a factor which ensures that $\sum_{j=1}^n a_{ij} \geq 1$, for $i = 1, \dots, m$, and that at least one of these inequalities is tight. This ensures feasibility because $x = (1, \dots, 1)$ satisfies all constraints. However, the generation scheme fails to satisfy the Unbiasedness and Completeness properties because $x = (1, \dots, 1)$ is always feasible and because $\sum_{j=1}^n a_{ij} = 1$ for at least one $i \in \{1, \dots, m\}$. A less efficient but also less biased approach would be to generate both feasible and infeasible instances, and then discard the latter. This is an example of a tradeoff between the Efficiency property and the Unbiasedness and Completeness properties.

4.3.5 Routing

One of the most extensively studied optimization problems for which synthetic data is needed is the traveling salesman problem (Arthur and Frendewey 1988). The intractability of this problem (Karp 1972) makes it difficult to find optimal solutions to be used as benchmarks for comparison with heuristic solution procedures. Several possible solutions to this problem are discussed by McGeoch (1996). One possible approach is to generate test problems for which the optimal solution and its value are known by construction. In Arthur and Frendewey (1988), this is achieved by relaxing the subtour elimination constraints in the problem. An optimal solution to the resulting assignment model is used to construct both an optimal solution to the traveling salesman problem and a randomized data set for which that solution is optimal. The authors compare the solution difficulty of their problems with purely random problems, based on their solvability using a standard optimization routine. They conclude that their problems are as hard as purely random ones. However, the proposed procedure fails to satisfy the Completeness property, since the subtour elimination constraints are redundant in all of the data sets they generate. This may lead to unrealistically good performance by some solution procedures, and unrealistically poor performance by others, thus violating the Unbiasedness property. Hence, a detailed computational study would be needed to reveal whether the bias has influenced the results. Moreover, conducting such a study would be approximately as computationally burdensome as solving the intractable optimization problem itself.

An interesting variant known as the black and white traveling salesman problem is studied by Ghiani et al. (2006). For this problem, a tour is feasible only if the number of consecutive white nodes it visits and the tour length between any pair of black nodes both meet prespecified upper bound constraints. Applications of this problem arise in aircraft maintenance. The data generation scheme used by Ghiani et al. (2006) generates the black nodes uniformly among all of the white nodes. It seems unlikely that this is representative of most practical situations. Similar comments can be made about the uniformly generated locations of the pick-up and drop-off locations in a study of the dial-a-ride problem (Cordeau 2006). Consequently, in both research studies, failure to satisfy the Completeness property may be an issue.

Interesting data generation issues arise when sampling from library test data. Recall from the Sect. 4.1 that the library data itself may be flawed. Nonetheless, using this approach, sampling needs to be representative, in order to satisfy the Comparability property. This approach is illustrated by Verweij et al. (2003) to test a sample average approximation method for stochastic routing problems. For each data set, a preliminary graph is chosen from a test bank of graphs. Then, each time a node of the graph is chosen, it is connected to a given number of nodes to which it has not previously been connected. Finally, a source and sink are randomly chosen from among the pairs of nodes that have the maximum value of the minimum number of arcs on any path between them. This approach introduces no additional bias be-

yond that which may already exist in the library data. It also ensures Completeness through parametric variation of the number of new nodes connected each time.

A computational study of an algorithm for routing in large-scale multicommodity telecommunications networks, discussed by Larsson and Yuan (2004), involves both library and synthetic data. Since different types of data are being used, an important requirement of the generation scheme is to satisfy the Comparability property. The authors address this requirement by generating their random planar networks with a topology that emulates the typical structure of a telecommunications network. The nodes are randomly generated as points in the plane. The arcs are then created in increasing order of Euclidean distance, so that the resulting network is planar. The commodities are defined by randomly chosen node pairs. The arc cost between a pair of nodes is defined by the Euclidean distance between them. This is a reasonable simplification considering the context of the authors' application and experiment.

An important issue in designing a synthetic data generation scheme for vehicle routing problems is integrality of the arc costs. Various procedures for identifying upper and lower bound solutions use truncation, as discussed by Toth and Vigo (2002). Hence, the extent to which the initial data is truncated influences the results. Consequently, the Comparability property may not be satisfied. An example occurs in comparing the results obtained by Fisher (1994) for real-valued Euclidean cost matrices with those obtained by Miller (1995) for Euclidean costs rounded to the nearest integer.

4.3.6 Data Mining

An empirical comparison of the performance of four widely used clustering algorithms, with applications to data mining and knowledge discovery, is considered by Wei et al. (2003). The four algorithms are CLARA (Kaufman and Rousseeuw 1990), CLARANS (Ng and Han 1994), and GAC-R³ and GAC-RAR_w (Estivill-Castro and Murray 1997). The synthetic data generation scheme by Wei et al. (2003) allows for variations in data size, number of clusters, cluster distributions, cluster asymmetry, and data randomness. Several constraints are imposed on the generation process. First, all clusters are assumed to have the same radius. Also, clusters are assumed to be randomly and evenly divided into two groups, with all clusters in the same group having the same cluster size. Finally, the number of objects in both smaller and larger clusters is specified by a formula. The introduction of these constraints into the data generation process raises several concerns. First, it is unclear whether biases have been introduced, thereby violating the Unbiasedness property. Second, it is unclear whether the constraints introduce variations in the data that are not present in real-world data, thereby violating the Parsimony property. Third, it may be difficult to satisfy the Completeness property because of the restrictions on data variety imposed by the constraints.

An alternative data generation scheme for clustering and outlier analysis is described by Pei and Zañane (2006). The objective is to generate clustered data points

in two or higher dimensional space. One important benefit of this work, relative to the generation scheme by Karyapis et al. (1999), is the detailed explanation of the steps, which documents Describability and promotes replication. The scheme starts with one of several simple probability distributions for the data in each cluster. Next, clusters with different shapes and densities are created using linear transformations, as well as linear equations to control line-based clusters and circle equations to control curve-shaped clusters. The flexibility of this data generation scheme provides Completeness. The generation scheme explicitly generates data sets with five different levels of clustering difficulty and three different levels of outliers, which again satisfies the Completeness property. The Efficiency property is demonstrated through the generation of all data sets with one million points in less than three seconds. The properties of Unbiasedness and Comparability are also satisfied by the generation scheme. It should be noted that the absence of formal constraints creates a data generation environment that is more flexible than many others in the optimization area.

4.3.7 Stochastic Programming

There are two issues in test problem generation for stochastic programming: how to generate the problem data, and how to generate the scenario tree. A popular test problem generator for stochastic programming data is SLP-IOR (Kall and Mayer 1993). This generator includes many options for generating univariate and multivariate, and discrete and continuous distributions. Also, the generator provides the facility to transform between a recourse model, a chance-constrained model, and a deterministic equivalent model, using the same data set. The high level of flexibility provided by SLP-IOR allows for many options in test problem design, and thereby supports the Completeness property. However, there is a lack of published guidelines for choosing from among the many possible problem instances. Perhaps as a consequence, many research studies in stochastic programming use library data instead of generating synthetic data. This avoids some of the problems in satisfying the Describability and Efficiency properties for this application.

There is a more substantial literature on scenario tree generation in multistage stochastic programming. As discussed by Heitsch and Römis (2005), the main challenge of scenario tree generation is finding a suitable compromise between a good approximation of the underlying probability distribution and the dimension of the stochastic model. The generation approach recommended by the authors minimizes a metric defined on the space of probability distributions. As a result, it satisfies the Unbiasedness property. Conditions are developed by Pennanen (2005) to determine when solutions to a sequence of finer discretizations converge to an optimal solution. This result provides conditions that can be used to satisfy the Unbiasedness property.

Two minimal criteria for scenario tree generation are established by Kaut and Wallace (2003). The first, stability, requires that the same optimal value arises from

different scenario trees. The second is that the scenario tree should not introduce any bias relative to the true solution. Further, an approach is proposed for testing scenario generation methods. Several techniques for generating scenario trees that allow interstage dependencies are discussed by Dupačová et al. (2000). These techniques include cluster analysis and importance sampling. A scenario generation heuristic that satisfies the Unbiasedness and Efficiency properties is given by Høyland et al. (2003). The application of nonlinear programming techniques to generate a limited number of discrete outcomes that satisfy prespecified statistical properties is studied by Høyland and Wallace (2001). Also, Dupačová et al. (2003) discuss how a given scenario tree can be reduced to a scenario subset of prespecified cardinality and a probability measure based on this set. Hence, test instances can be generated based on the size of the reduced scenario tree, rather than the original tree. Since many more of the reduced scenario trees can be generated, this supports the Completeness and Efficiency properties. The issue of correlation within the random data (see Sect. 4.3.1) is shown to be potentially significant in an application discussed by Lium et al. (2007). However, no guidelines are provided for incorporating correlation into a data generation scheme for stochastic programming.

Finally, we mention a decomposition-based branch-and-bound algorithm for two-stage stochastic programs that is described by Sherali and Zhu (2007). A computational study is conducted using specific formulations for both the first-stage and the second-stage problems. This raises concerns about whether the Completeness property is being satisfied. In this experiment, there is a trade-off between Completeness and Describability. If the first-stage and the second-stage problem formulations are varied, as the Completeness property suggests, then a description of the data, i.e., the full details of each formulation, is not concise.

4.3.8 Intractable Problems

Knowledge of an optimal solution is frequently required to satisfy the purposes of a given experiment. For many intractable problems, obtaining this information for a set of instances can be very time consuming or even impossible. As a result, some researchers have developed data generation approaches that avoid this difficulty. One approach is to use library data for which optimal solutions are known. Another approach is to generate problem instances with known optimal solutions (Pilcher and Rardin 1992). The proposed procedure is based on obtaining a partial description of the polytope of solutions and then generating random cuts. Once the problem has been generated, both the optimal solution and the form of valid inequalities that is required to solve the problem using cutting planes are known. Starting from a purely random problem and a randomly selected feasible solution, a random sample of tight valid inequalities is selected. This sample, along with dual variable information, is used to compute a cost vector that ensures optimality of the selected solution. This approach is illustrated by generating instances of the asymmetric traveling salesman problem. An extension to the time-dependent traveling salesman

problem is described by Vander Wiel and Sahinidis (1995). As observed by Rardin and Uzsoy (2001), the polyhedral approach by Pilcher and Rardin (1992) relies on the availability of problem-specific information, in particular detailed knowledge of the polyhedral structure of the problem. There is empirical evidence that problems generated by this approach are not particularly easy to solve using most heuristics. The potential concerns about failure to satisfy the Unbiasedness and Completeness properties are similar to those in Arthur and Frendewey (1988), as discussed in Sect. 4.3.5. However, the data generation scheme proposed by Pilcher and Rardin (1992) introduces greater randomness into the test problems generated, which mitigates concerns about failure to satisfy the Completeness property.

An important issue with respect to the Completeness property is the generation of data sets with a variety of levels of solution difficulty. This is particularly challenging for an optimization problem that is *NP*-hard (Garey and Johnson 1979). This is because, for many such problems, typical problem instances are easy to solve; however, difficult problem instances occur at critical values of certain problem parameters (Cheeseman et al. 1991). The parameter values where such instances occur are known as *phase transitions*. On one side of the phase transition, there is an underconstrained region where a high density of solutions makes them easy to find. On the other side of the phase transition, there is an overconstrained region with fewer solutions, but where the few solutions that do exist are easy to find (Bailey et al. 2007). Consequently, the hardest problem instances occur between these two regions. Knowledge of where phase transitions occur facilitates the design of a data generation scheme that satisfies the Completeness property. This issue is explored for graph coloring problems in Culberson et al. (1995). The authors identify classes of *k*-colorable graphs for which various parameters accurately characterize the location of the phase transition. This issue is further explored for the graph bipartitioning problem by Angel and Zissimopoulos (1998). The authors show that the performance of a local search procedure can be predicted from a measure called the autocorrelation coefficient, the maximum distance between any two solutions, and the size of the neighborhood. Angel and Zissimopoulos (2000) extend this idea to describe a hierarchy of *NP*-hard combinatorial optimization problems, based on their autocorrelation coefficients. This hierarchy can be used to evaluate and support the property of Comparability between research studies.

A computational study of a single-phase interior-point algorithm for the monotone complementarity problem is described by Andersen and Ye (1998). One of the features that is tested is how efficiently the algorithm detects an infeasible solution. This raises the issue of how to generate infeasible problem instances. The approach used in Andersen and Ye (1998) sets the right-hand-side value of one of the constraints to be one less than the minimum possible amount attainable if all variables are set to their lower bounds. Instances generated in this way are clearly infeasible. A disadvantage of this scheme, however, is that it makes the right-hand-side value dependent on all of the variable lower bounds, many of which may be redundant in the solution. This fails to satisfy the Parsimony property. Because only one constraint is violated, it also fails to satisfy the Consistency property. A further concern is that the right-hand-side value is always set by using a difference of one. Since

the resulting instances are “only just infeasible”, this approach generates problem instances for which feasibility may be hardest to detect. However, if the experiment is indeed to estimate how efficiently the algorithm detects an infeasible solution, then the Completeness property suggests that a variety of values for this difference should be used. Moreover, the data generation scheme favors those procedures that efficiently detect minimally infeasible problems, and therefore fails to satisfy the Unbiasedness property.

4.4 Concluding Remarks

We recommend the following general protocol for the generation of synthetic data. First, determine the features of the application and the features of the experiment. Then, use the features of the application and the experiment, along with the generation properties, to develop a data generation scheme. Next, determine the attributes that the data should possess. After the data is generated, measure these attributes to validate the data. We believe that this protocol can be applied quite generally. Possible applications include the various applications of synthetic data generation discussed in the “Introduction”.

There are several opportunities for further research related to synthetic data generation in optimization. First, the general protocol which we describe can be specified in greater detail for many classes of optimization problems, following the example by Hall and Posner (2001) for scheduling problems. This will provide a valuable service to researchers, both by standardizing data generation procedures and by eliminating the more problematic practices that currently exist. Also, there are several data generation issues which arise in optimization that pose unusual challenges. Examples include the generation of infeasible problems (Andersen and Ye 1998) and the generation of data for assessing solution quality in stochastic programs (Bayraksan and Morton 2007). These applications of data generation have not been extensively discussed in the literature, and standard procedures need to be developed for them.

Acknowledgements This research is supported by the Summer Fellowship Program of the Fisher College of Business, The Ohio State University, to the first author. Helpful comments were provided by Tito Homem-de-Mello and two anonymous reviewers.

References

- Aksoy H, Bayazit M (2000) A model for daily flows of intermittent streams. *Hydrological Processes* 14:1725–1744
- Amini MM, Racer M (1994) A rigorous computational comparison of alternative solution methods for the generalized assignment problem. *Management Science*

40:868–890

- Andersen ED, Ye Y (1998) A computational study of the homogeneous algorithm for large-scale convex optimization. *Computational Optimization and Applications* 10:243–269
- Angel E, Zissimopoulos V (1998) Autocorrelation coefficient for the graph bipartitioning problem. *Theoretical Computer Science* 191:229–243
- Angel E, Zissimopoulos V (2000) On the classification of *NP*-complete problems in terms of their correlation coefficient. *Discrete Applied Mathematics* 99:261–277
- Arthur JL, Friendewey JO (1988) Generating travelling-salesman problems with known optimal tours. *Journal of the Operational Research Society* 39:153–159
- Atamtürk A (2007) Strong formulations of robust mixed 0-1 programming. *Mathematical Programming* 108:235–250
- Bailey DD, Dalmau V, Kolaitis PG (2007) Phase transitions of PP-complete satisfiability problems. *Discrete Applied Mathematics* 155:1627–1639
- Balas E, Martin CH (1980) Pivot and complement - a heuristic for 0-1 programming. *Management Science* 26:86–96
- Balas E, Zemel E (1980) An algorithm for large zero-one knapsack problems. *Operations Research* 28:1130–1154
- Bauer HU, Herrmann M, Villmann T (1999) Neural maps and topographic vector quantization. *Neural Networks* 12:659–676
- Bayraksan G, Morton D (2007) Assessing solution quality in stochastic programs. *Mathematical Programming, Series B* 108:495–514
- Bertsimas D, Natarajan K, Teo CP (2006) Persistence in discrete optimization under demand uncertainty. *Mathematical Programming* 108:251–274
- Beyer K, Goldstein J, Ramakrishnan R (1999) When is “nearest neighbour” meaningful? *Database Theory - ICDT '99* 1540:217–235
- Bienstock D, Raskina O, Saniee I, Wang Q (2006) Combined network design and multiperiod pricing: Modeling, solution techniques and computation. *Operations Research* 54:261–276
- Bijmolt THA, Wedel M (1999) A comparison of multidimensional scaling methods for perceptual mapping. *Journal of Marketing Research* 36:277–285
- Brahimi N, Dauzère-Pérès S, Najid NM (2006) Capacitated multi-item lot-sizing problems with time windows. *Operations Research* 54:951–967
- Cario MC, Clifford JJ, Hill RR, Yang J, Yang K, Reilly CH (2002) An investigation of the relationship between problem characteristics and algorithm performance: A case study of the GAP. *IIE Transactions* 34:297–312
- Chalmet L, Gelders L (1976) Lagrangean relaxation for a generalized assignment-type problem. North-Holland, Amsterdam, The Netherlands
- Cheeseman P, Kanefsky B, Taylor WM (1991) Where the *really* hard problems are. In: *Proceedings of IJCAI-91*, Morgan Kaufmann, San Mateo, CA, pp 331–337
- Chen ZL, Pundoor G (2006) Order assignment and scheduling in a supply chain. *Operations Research* 54:555–572
- Cordeau JJ (2006) A branch-and-cut algorithm for the dial-a-ride problem. *Operations Research* 54:573–586

- Culberson J, Beacham A, Papp D (1995) Hiding our colors. In: Proceedings of the CP '95 Workshop on Studying and Solving Really Hard Problems, Cassis, France, pp 31–42
- Degraeve Z, Schrage L (1997) Should I use a portable generator in an emergency? Working paper, Department of Applied Economic Sciences, Katholieke Universiteit Leuven, Belgium
- Demeulemeester E, Vanhoucke M, Herroelen W (2003) RanGen: A random network generator for activity-on-the-node networks. *Journal of Scheduling* 6:17–38
- Dupačová J, Consigli G, Wallace SW (2000) Scenarios for multistage stochastic programs. *Annals of Operations Research* 100:25–53
- Dupačová J, Gröwe N, Römisches W (2003) Scenario reduction in stochastic programming: An approach using probability metrics. *Mathematical Programming, Series A* 95:493–511
- Estivill-Castro V, Murray AT (1997) Spatial clustering for data mining with generic algorithms. Technical Report FIT-TR-97-10, Faculty of Information Management, Queensland University of Technology
- Fischetti M, Lodi A, Martello S, Toth P (2001) A polyhedral approach to simplified crew scheduling and vehicle scheduling problems. *Management Science* 47:833–850
- Fisher ML (1994) Optimal solution of vehicle routing problems using minimum k -trees. *Operations Research* 42:626–642
- Freed JA (2000) Conceptual comparison of two computer models of corpuscle sectioning and of two algorithms for correction of ploidy measurements in tissue sections. *Analytical and Quantitative Cytology and Histology* 22:17–25
- Frenje L, Juhlin C (1998) Scattering of seismic waves simulated by finite difference modelling in random media: Application to the Gravberg-1 well. Sweden *Tectonophysics* 293:61–68
- Garey MR, Johnson DS (1979) *Computers and Intractability: a Guide to the Theory of NP-Completeness*. W.H. Freeman, San Francisco, CA
- Gelius LJ, Westerdahl H (1997) Seismic noise modelling. *Journal of Seismic Exploration* 6:351–366
- Ghiani G, Laporte G, Semet F (2006) The black and white traveling salesman problem. *Operations Research* 54:366–378
- Gonzalez J, Gutierrez R (1999) Direct motion estimation from a range scan sequence. *Journal of Robotic Systems* 16:73–80
- Goutte C (2000) Extraction of the relevant delays in temporal modelling. *IEEE Transactions on Signal Processing* 48:1787–1795
- Grate JW, Wise BM, Abraham MH (1999) Method for unknown vapor characterization and classification using a multivariate sorption detector. *Analytical Chemistry* 71:4544–4553
- Guignard M, Rosenwein MB (1989) An improved dual based algorithm for the generalized knapsack problem. *Operations Research* 37:658–663
- Hadjar A, Marcotte O, Soumis F (2006) A branch-and-cut algorithm for the multiple depot vehicle scheduling problem. *Operations Research* 54:130–149

- Hall NG, Posner ME (2001) Generating experimental data for computational testing with machine scheduling applications. *Operations Research* 49:854–865
- Hall NG, Posner ME (2007) Performance prediction and preselection for optimization procedures. *Operations Research* 55:703–716
- Hariri AM, Potts CN (1983) An algorithm for single machine sequencing with release dates to minimize total weighted completion time. *Discrete Applied Mathematics* 5:99–109
- Hays WL (1973) *Statistics for the Social Sciences*, 2nd edn. Holt, Rinehart and Winston, Inc., New York, NY
- Heitsch H, Römisches W (2005) Generation of multivariate scenario trees to model stochasticity in power management. In: *Power Tech, IEEE Russia*, pp 1–7
- Herique A (1999) Radio wave back-propagating in polar coordinates: A linear filter in the time-frequency angle-frequency domain. *Radio Science* 34:509–519
- Hill RR, Reilly CH (1994) Composition for multivariate random variables. In: *Proceedings, 1994 Winter Simulation Conference*, Institute of Electrical and Electronics Engineers, Orlando, FL, pp 332–342
- Hill RR, Reilly CH (2000) The effects of coefficient correlation structure in two-dimensional knapsack problems on solution procedure. *Management Science* 46:302–317
- Ho TK, Baird HS (1997) Large-scale simulation studies in image pattern recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 19:1067–1079
- Hooker JN (1994) Needed: An empirical science of algorithms. *Operations Research* 42:201–212
- Hooker JN (1995) Testing heuristics: We have it all wrong. *Journal of Heuristics* 1:33–42
- Hooshyar MA, Lam TH, Razavy M (2000) Inverse problem of the wave equation and the Schwinger approximation. *Journal of the Acoustical Society of America* 107:404–413
- Høyland K, Wallace SW (2001) Generating scenario trees for multistage decision problems. *Management Science* 47:295–307
- Høyland K, Kaut M, Wallace SW (2003) A heuristic for moment-matching scenario generation. *Annals of Operations Research* 24:169–185
- Iman RL, Conover WJ (1982) A distribution-free approach to inducing rank correlation among input variables. *Communications in Statistics: Simulation and Computing* B11:311–334
- John TC (1989) Tradeoff solutions in single machine production scheduling for minimizing flow time and maximum penalty. *Computers & Operations Research* 16:471–479
- Kadlec RH (2000) The inadequacy of first-order treatment wetland models. *Ecological Engineering* 15:105–119
- Kall P, Mayer J (1993) SLP-IOR: On the design of a workbench for testing SLP codes. *Revista Investigación Operacional* 14:148–161
- Karp RM (1972) Reducibility among combinatorial problems. In: *Complexity of Computer Computations*, Plenum, New York, NY, pp 85–103

- Karyapis G, Han EH, Kumar V (1999) CHAMELEON: A hierarchical clustering algorithm using dynamic modeling. *IEEE Computer* 32:68–75
- Kaufman L, Rousseeuw PJ (1990) *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley, New York, NY
- Kaut M, Wallace SW (2003) Evaluation of scenario-generation methods for stochastic programming. Working paper, Molde University College, Norway
- Koehler JR, Owen AB (1996) Computer experiments. In: Ghosh S, Rao C (eds) *Handbook of Statistics*, vol 13, Elsevier Science, New York, NY, pp 261–308
- Kolisch R, Sprecher A, Drexel A (1995) Characterization and generation of a general class of resource-constrained project scheduling problems. *Management Science* 41:1693–1703
- Krieger AM, Green PE (1999) A cautionary note on using internal cross validation to select the number of clusters. *Psychometrika* 64:341–353
- Laguna M, Rafael M (2001) A GRASP for coloring sparse graphs. *Computational Optimization and Applications* 19:165–178
- Larsson T, Yuan D (2004) An augmented Lagrangian algorithm for large scale multicommodity routing. *Computational Optimization and Applications* 27:187–215
- Law AM, Kelton WD (1991) *Simulation Modeling and Analysis*, 2nd edn. McGraw-Hill, New York, NY
- Linderöth J, Shapiro A, Wright S (2006) The empirical behavior of sampling methods for stochastic programming. *Annals of Operations Research* 142:215–241
- Lium AG, Crainic TG, Wallace SW (2007) Correlations in stochastic programming: A case from stochastic service network design. *Revista Investigación Operacional* 24:161–179
- Lu Z, Wyss M, Pulpan H (1997) Details of stress directions in the Alaska subduction zone from fault plane solutions. *Journal of Geophysical Research-Solid Earth* 102:5385–5402
- Martello S, Toth P (1979) The 0-1 knapsack problem. In: Christofides N, Mingozzi A, Toth P, Sandi C (eds) *Combinatorial Optimization*, Wiley, New York, NY, pp 237–279
- Martello S, Toth P (1981) An algorithm for the generalized assignment problem. In: Brans JP (ed) *Operational Research '81*, North-Holland, Amsterdam, The Netherlands, pp 589–603
- Martello S, Toth P (1988) A new algorithm for the 0-1 knapsack problem. *Management Science* 34:633–644
- Martello S, Toth P (1997) Upper bounds and algorithms for hard 0-1 knapsack problems. *Operations Research* 45:768–778
- McGeoch CC (1996) Towards an experimental method for algorithm simulation. *INFORMS Journal on Computing* 8:1–15
- McIntosh SW, Charbonneau P, Brown JC (2000) Preconditioning the differential emission measure (T-e) inverse problem. *Astrophysics Journal* 529:1115–1130
- Miller DL (1995) A matching based exact algorithm for capacitated vehicle routing problems. *ORSA Journal on Computing* 7:1–9

- Munizaga MA, Heydecker BG, Ortuzar JD (2000) Representation of heteroskedasticity in discrete choice models. *Transportation Research B - Methodology* 34:219–240
- Ng R, Han J (1994) Efficient and effective clustering methods for spatial data mining. In: *Proceedings of International Conference on Very Large Data Bases*, Santiago, Chile, pp 144–155
- Ow P (1985) Focused scheduling in proportionate flowshops. *Management Science* 31:852–869
- Palmer CR, Faloutsos C (2000) Density biased sampling: An improved method for data mining and clustering. *SIGMOD Record* 29:82–92
- Pan Y, Shi L (2007) On the equivalence of the max-min transportation lower bound and the time-indexed lower bound for single-machine scheduling problems. *Mathematical Programming* 110:543–559
- Patterson JH (1984) A comparison of exact procedures for solving the multiple-constrained resource project scheduling problem. *Management Science* 30:854–867
- Pearson WR, Robins G, Zhang TT (1999) Generalized neighbor-joining: more reliable phylogenetic tree reconstruction. *Molecular Biology and Evolution* 16:806–816
- Pei Y, Zaïane O (2006) A synthetic data generator for clustering and outlier analysis. In: *Technical report TR06-15*, University of Alberta, Edmonton, Alberta
- Pennanen T (2005) Epi-convergent discretizations of multistage stochastic programs. *Mathematics of Operations Research* 30:245–256
- Pilcher MG, Rardin RL (1992) Partial polyhedral description and generation of discrete optimization problems with known optima. *Naval Research Logistics* 39:839–858
- Potts CN, Van Wassenhove LN (1985) A Lagrangean based branch and bound algorithm for single machine sequencing with precedence constraints to minimize total weighted completion time. *Management Science* 31:1300–1311
- Potts CN, Van Wassenhove LN (1988) Algorithms for scheduling a single machine to minimize the weighted number of late jobs. *Management Science* 34:843–858
- Potts CN, Van Wassenhove LN (1992) Single machine scheduling to minimize total late work. *Operations Research* 40:586–595
- Qin G, Jing BY (2000) Asymptotic properties for estimation of partial linear models with censored data. *Journal of Statistical Planning and Inference* 84:95–110
- Racer M, Amini MM (1994) A robust heuristic for the generalized assignment problem. *Annals of Operations Research* 50:487–503
- Rardin RL, Uzsoy R (2001) Experimental evaluation of heuristic optimization algorithms: A tutorial. *Journal of Heuristics* 7:261–304
- Ray KS, Ghoshal J (2000) Neuro-genetic approach to multidimensional fuzzy reasoning for pattern classification. *Fuzzy Sets and Systems* 112:449–483
- Reiter JP (2002) Satisfying disclosure restrictions with synthetic data sets. *Journal of Official Statistics* 18:531–543
- Romeijn HE, Morales DR (2001a) Generating experimental data for the generalized assignment problem. *Operations Research* 49:866–878

- Romeijn HE, Morales DR (2001b) A probabilistic analysis of the multi-period single-sourcing problem. *Discrete Applied Mathematics* 112:301–328
- Ross GT, Soland RM (1975) A branch and bound algorithm for the generalized assignment problem. *Mathematical Programming* 8:91–103
- Ross GT, Soland RM (1977) Modeling facility location problems as generalized assignment problems. *Management Science* 24:345–357
- Roversi P, Irwin JJ, Bricogne G (1998) Accurate charge density studies as an extension of Bayesian crystal structure determination. *Acta Crystallographica Section A* 54:971–996
- Ruchala KJ, Olivera GH, Schloesser EA (1999) Megavoltage CT on a tomotherapy system. *Physics in Medicine and Biology* 44:2597–2621
- Rushmeier RA, Nemhauser GL (1993) Experiments with parallel branch-and-bound algorithms for the set covering problem. *Operations Research Letters* 13:277–285
- Schaffer C (1994) A conservation law for generalization performance. In: *International Conference on Machine Learning*, Morgan Kaufmann, San Francisco, CA, pp 259–265
- Schena G, Chiaruttini C (2000) A stereologically posed mass balance for calculating the distributed efficiency of particle separation systems. *International Journal of Mineral Processing* 59:149–162
- Schwindt C (1995) A new problem generator for different resource-constrained project scheduling problems with minimal and maximal time lags. *WIOR-Report-449*, Institut für Wirtschaftstheorie und Operations Research, University of Karlsruhe
- Shen L, Shen H, Cheng L (1999) New algorithms for efficient mining of association rules. *Information Sciences* 118:251–268
- Sherali HD, Smith JC (2006) A polyhedral study of the generalized vertex cover problem. *Mathematical Programming* 107:367–390
- Sherali HD, Zhu X (2007) On solving discrete two-stage stochastic programs having mixed-integer first- and second-stage variables. *Mathematical Programming* 105:597–616
- Smith-Miles KA (2008) Cross-disciplinary perspectives on meta-learning for algorithm selection. *ACM Computing Surveys* 41:6.1–6.25
- Smith-Miles KA, James RJW, Giffin JW, Tu Y (2009) A knowledge discovery approach to understanding relationships between scheduling problem structure and heuristic performance. In: *Learning and Intelligent Optimization Conference (LION 3)*, Trento, Italy
- Toth P, Vigo D (2002) Models, relaxations and exact approaches for the capacitated vehicle routing problem. *Discrete Applied Mathematics* 123:487–512
- Trick MA (1992) A linear relaxation heuristic for the generalized assignment problem. *Naval Research Logistics* 39:137–151
- Uma RN, Wein J (1998) On the relationship between combinatorial and LP-based approaches to NP-hard scheduling problems. *Integer Programming and Combinatorial Optimization, Lecture Notes in Computer Science* 1412:394–408

- Vander Wiel RJ, Sahinidis NV (1995) Heuristic bounds and test problem generation for the time-dependent traveling salesman problem. *Transportation Science* 29:167–183
- van de Velde SL (1995) Dual decomposition of a single-machine scheduling problem. *Mathematical Programming* 69:413–428
- Verweij B, Ahmed S, Kleywegt A, Nemhauser G, Shapiro A (2003) The sample average approximation method applied to stochastic routing problems: A computational study. *Computational Optimization and Applications* 24:289–333
- Wei CP, Lee YH, Hsu CM (2003) Empirical comparison of fast partitioning-based clustering algorithms for large data sets. *Expert Systems with Applications* 24:351–363
- Wilson RC, Hancock ER (2000) Bias variance analysis for controlling adaptive surface meshes. *Computer Vision and Image Understanding* 77:25–47
- Wolpert DH, Macready WG (1997) No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation* 1:67–82
- Wu D, Golbasi H (2004) Multi-item, multi-facility supply chain planning: Models, complexities and algorithms. *Computational Optimization and Applications* 28:325–356
- Xu S, Freund RM, Sun J (2003) Solution methodologies for the smallest enclosing circle problem. *Computational Optimization and Applications* 25:283–292
- Yaman H, Karahan OE, Pinar MÇ (2007) Restricted robust uniform matroid maximization under interval uncertainty. *Mathematical Programming* 110:431–441
- Yuval (2000) Neural network training for prediction of climatological time series, regularized by minimization of the generalized cross validation function. *Monthly Weather Review* 128:1456–1473